FORENSIC CODEBASE AUDIT · READ-ONLY

How MTB turns a protocol PDF into a patient schedule

SCOPE	BACKEND LOC	TESTS RUN	DATE
MTB-APP	~13.3k UCTSM	590 / 84 files	2026-09-05

THE FINDING

The adapter between the live extraction pipeline and the UCTSM engine **already exists, is wired into the Add Trial screen, and passes 35 tests.** The question is not what to build — it is why the path does not complete for a real protocol. Four reasons, three of them hard blockers.

B1 Patient enrolment fails unless the first anchor is literally named “Baseline” — P0

The bridge requires an anchor coded or typed `BASELINE`; the adapter’s anchor-type map cannot produce one. An anchor named “First Dose” or “Randomisation” yields no baseline and enrolment raises — leaving the patient with no schedule at all, because the canonical path deliberately refuses to fall back to legacy. The journey test hides this by naming its fixture anchor “Baseline”.

- PROVEN BY PROBE · THREE ANCHOR NAMES, ONE PLAN

B2 Every footnote blocks approval, and nothing can resolve one — P0

Each footnote, condition and unclear window becomes an `UNRESOLVED_QUALIFIER`, which is blocking — correct by design, and asserted by its own test. But no endpoint and no UI can mark a qualifier resolved, so any realistic protocol produces a draft that can never reach `APPROVED`.

B3 The two projections disagree on which anchor is day zero — P0

The flat legacy view prefers a randomisation / first-dose anchor; the adapter takes whichever anchor came first. The same unanchored “Day 8” row resolves to two different dates depending on which surface you read.

- PROVEN BY PROBE · RANDOMISATION VS INFORMED CONSENT

B4 The extraction has never been graded against a real Schedule of Assessments — P1

The corpus eval is blocked on API billing, and its own findings file retracts the earlier “4/4 archetypes pass” figure as coming from synthetic PDFs built to match the prompt’s assumptions. This is an **extraction validation gap, not a code gap** — and it is procurement time, not engineering time, so it should start now.

SECTIONS

Section 1

Executive Summary

MTB has **one** live protocol-extraction pipeline, **one** canonical schedule engine, and — contrary to what the task premise assumed — **the adapter between them already exists, is wired into the Add Trial screen, and is covered by 35 passing tests.**

The production path today is:

```
Protocol PDF (Add Trial screen)
  → POST /api/protocols/extract                    (Gemini + LangGraph, in-memory only)
  → ExtractedSchedule{canonical_plan: CanonicalSchedulePlan v2}
  → Mongo db.protocol_extractions            (2-hour TTL cache)
  → POST /api/trials                          (trial row created)
  → POST /api/trials/{id}/protocol-extractions/{eid}/consume
  → Mongo db.schedule_definitions            (durable, status=draft_review)
  → POST /api/trials/{id}/uctsm-schedule
  → app/services/canonical_import.py          ← THE ADAPTER. It exists.
  → app/services/extraction_service.py::complete
  → SQL uctsm_schedule_versions              (status = VALIDATION_REQUIRED)
  → sponsor/protocol-schedule screen          (validate → submit → confirm → approve)
  → POST /api/patients → OperationalBridgeService.enroll_patient
  → UCTSM evaluation → projected back into Mongo db.visit_instances
```

So the question is not "what adapter do we need to build". It is **"why does this path not complete for a real protocol"**. The audit found four reasons, three of them hard blockers:

There is also a **second, unconnected extraction pipeline** (`app/extraction/` , Claude-backed, 26 stages, with the footnote/qualifier and confinement stages the live one lacks). Its API exists, its frontend wrapper functions exist, and **nothing calls them.** It cannot run on Add Trial protocols anyway, because the live flow never stores the PDF.

What must NOT be rebuilt: the extraction pipeline, the canonical plan schema, the adapter, the schedule engine, the review/approval lifecycle, versioning, patient assignment, impact preview, deviations, the parity/read-mode cutover gate, and the UCTSM screens. All of that is implemented and, apart from the blockers, connected.

Section 2

Repository Architecture

Single repo, no monorepo tooling. Not a git repository at the audit root; the app itself is (`MTB-APP/.git`).

```
MTB-APP/
├── backend/                      FastAPI, Python 3.11
│   ├── server.py                 481 KB — the operational monolith. 154 routes,
│   │                             all Mongo. Auth, orgs, trials, legacy visit
│   │                             schedule, patients, chat, files, reports, and
│   │                             the Mongo↔UCTSM bridge helpers.
│   ├── admin_routes.py  (102 KB) platform-admin router
│   ├── org_routes.py     (46 KB) org-admin router
│   ├── protocol_extraction.py (101 KB)  LIVE extraction: models + 4 providers
│   ├── protocol_agent.py     (114 KB)  LIVE extraction: the LangGraph graph
│   ├── schedule_schema.py     (72 KB)  CanonicalSchedulePlan v2 + projection
│   ├── protocol_document_index.py     PDF → page index, chunking, retrieval
│   ├── storage.py                     local-disk / S3 file storage
│   ├── otp_service.py, google_places.py
│   ├── app/                      THE UCTSM SUBSYSTEM (SQL, 13.3 kLOC)
│   │   ├── api/uctsm.py          41 routes, mounted at /api/uctsm
│   │   ├── db/                   base.py (engine), models.py (39 tables),
│   │   │                         repositories.py (draft persist/read)
│   │   ├── domain/schedule/      the engine: models, timing, evaluator,
│   │   │                         conditional, condition, validator, deviation,
│   │   │                         diff, parity, projection, enrolment,
│   │   │                         anchor_status
│   │   ├── extraction/           SECOND extractor (Claude): graph, prompts,
│   │   │                         claude_provider, llm_client, completeness, runner
│   │   └── services/             canonical_import (ADAPTER), canonical_bridge,
│   │                             extraction_service, schedule_service,
│   │                             impact_service, parity_service,
│   │                             operational_bridge/projection/read,
│   │                             schedule_projection, notification_projection,
│   │                             deviation_service, dashboard_service,
│   │                             anchor_status_service, demo_service
│   ├── alembic/versions/         13 migrations (0001…0013)
│   ├── eval/                     extraction eval harness + FINDINGS.md
│   ├── tests/                    84 test files
│   └── uctsm-dev.db              SQLite — the dev UCTSM database
├── frontend/                     Expo / React Native (expo-router), TS
│   ├── app/(app)/…               file-routed screens by role
│   └── src/features/uctsm/       the UCTSM UI: api.ts, presentation.ts,
│                                 ScheduleTable, ProtocolSchedule/Review,
│                                 PatientSchedule, ScheduleVersions,
│                                 ActionBoard, Workbench
└── docs/                         17 prior analysis documents
```

Two databases, split by concern.

- **MongoDB Atlas** (`MONGO_URL` , `DB_NAME=mtb_app`) — everything operational: identity, orgs, trials, the legacy visit schedule, patients, `visit_instances` , chat, files, notifications, audit log. 49 collections.

- **SQL via SQLAlchemy** (`UCTSM_DATABASE_URL`) — the canonical schedule engine, 39 `uctsm_*` tables. **Currently pointed at `sqlite:///uctsm-dev.db`** with `UCTSM_DEMO_MODE=true` , which makes `app/db/base.py::build_session_factory` call `Base.metadata.create_all()` and bypass Alembic entirely. Alembic + PostgreSQL is the documented production target (`app/db/base.py:24-27`).

No workers, no queues, no cron. Background work is `BackgroundTasks` and `asyncio.create_task` in `server.py` 's startup hook only.

Section 3

Current Protocol PDF Extraction Architecture

Provider is selected at call time by `protocol_extraction.get_extractor()` from `PROTOCOL_EXTRACTION_PROVIDER` . Four implementations exist:

PROVIDER	CLASS	PIPELINE	NOTES
gemini (default, and what `.env` sets)	`GeminiProtocolExtractor`	full LangGraph agent	only provider with `extract_bundle_all`
claude	`ClaudeProtocolExtractor`	single call + repair	legacy fallback, no graph
openrouter	`OpenRouterProtocolExtractor`	single call	
ollama	`OllamaProtocolExtractor`	page-batch evidence + reduce	local, renders pages to images

Live config: `PROTOCOL_EXTRACTION_PROVIDER=gemini` , `PROTOCOL_EXTRACTION_MODEL=…-flash` , `MAX_REFINEMENTS=2` , `MIN_CONFIDENCE=0.75` , `MAX_PDF_BYTES=25 MB` , `MAX_OUTPUT_TOKENS=24000` .

Document handling (`protocol_document_index.py`)

- Text extraction: **pypdfium2** (`_extract_pdf_text_pages:171`). 1-based page numbers preserved so every cited evidence page is openable by a reviewer.
- **No OCR library.** `_text_status:162` classifies each page `text | sparse_text | image_or_empty` ; a page with no embedded text is rendered to the model as `[NO EMBEDDED TEXT — USE PDF VISION/OCR]` (`render_page_chunk:706`) and the multimodal model reads the attached PDF directly. OCR is delegated to the model, not performed.
- **Chunking:** `chunk_protocol_pages:628` — fixed **22 CORE pages** with **4 pages overlap** (`PROTOCOL_EVIDENCE_CHUNK_CORE_PAGES` / `…OVERLAP_PAGES`). Every page is in exactly one chunk's CORE set, so full coverage is structural, not heuristic. This deliberately replaced a keyword-scored ~24-page excerpt (see `evidence_sweep_node` 's docstring, `protocol_agent.py:1471`).
- **Cost control:** `GeminiProtocolExtractor._create_pdf_cache:1390` uploads the PDF once per extraction as a Gemini context cache (≥100 KB, 1800 s TTL) and every stage references it. Any cache failure silently falls back to per-call attachment (`_cached_generate:1437`) — caching can never fail an extraction.
- **Retries:** two layers. `_generate` retries a schema-invalid structured response once with a repair instruction (`protocol_extraction.py:1281`); `run_stage` retries a failed stage up to 3 times with exponential backoff and **checkpoints each completed stage** so a resumed run never re-pays for finished AI work (`protocol_agent.py:1375-1435`).

The PDF is never persisted. `extract_protocol_alias` reads the upload into memory and discards it. `canonical_bridge.plan_fingerprint` exists precisely because of this: the protocol version is content-addressed by the SHA-256 of the extracted plan, and `document_uri` is set to the non-resolvable `private://protocol-extractions/{id}` .

What the extraction reads: the complete document. Classification and the discovery map guide *prompting*, but the evidence sweep reads every page. Footnotes, appendices, narrative dosing sections and cross-referenced treatment plans are all in scope and explicitly instructed (`protocol_agent.py:393, 436, 631, 659, 805, 827`).

Section 4

Exact Extraction Call Flow

The graph, verified from `build_schedule_extraction_graph` (`protocol_agent.py:1365-1900`):

```
START → classify ──(>1 schedule option & none selected)──→ needs_selection → finalize → END
              └──→ discover ──(classifier AND map both say none)──→ no_schedule → finalize
                        └──→ evidence_sweep (N parallel chunk calls) → synthesize → audit
                                    audit ──accepted & no deterministic issues──→ finalize
                                    audit ──refinements < max──→ refine → audit
                                    audit ──otherwise──→ finalize
```

STEP	FILE	FUNCTION / CLASS	INPUT	OUTPUT	NEXT
1	`frontend/app/(app)/sponsor/add-trial.tsx:209`	`runExtract`	PDF asset	multipart POST	API
2	`backend/server.py:2218`	`extract_protocol_alias`	`UploadFile`	validates PDF, ≤25 MB	`pe.extract_protocol_bundle_all`
3	`protocol_extraction.py:2079`	`extract_protocol_bundle_all`	bytes	dispatch	`GeminiProtocolExtractor.extract_bundle_all`
4	`protocol_extraction.py:1567`	`extract_bundle_all`	bytes	page index + PDF cache	`run_protocol_extraction_agent_for_all_options`
5	`protocol_agent.py:2087`	`run_protocol_extraction_agent_for_all_options`	bytes, generate	fan-out per schedule option, ≤3 concurrent	`_run_schedule_extraction_graph`
6	`protocol_agent.py:1436`	`classify_node`	PDF	`DocumentTaskClassification` (doc type, task, `schedule_options[]` , evidence)	route

STEP	FILE	FUNCTION / CLASS	INPUT	OUTPUT	NEXT
7	`protocol_agent.py:1456`	`discover_node`	classification	`ScheduleDocumentMap` (where the schedule lives)	route
8	`protocol_agent.py:1471`	`evidence_sweep_node`	page chunks	`ScheduleChunkEvidence` × N → merged `ScheduleTimingEvidence` + `ScheduleVisitEvidence` (incl. `table_footnotes`)	synthesize
9	`protocol_agent.py:1637`	`synthesize_node`	all evidence	`ExtractedSchedule` — **provider actually returns `CanonicalScheduleResponse`**, i.e. `canonical_plan` only, no flat rows (`protocol_extraction.py:1266`)	audit
10	`protocol_agent.py:1652`	`audit_node`	expanded candidate	`ScheduleAudit` + deterministic issues from `_validate_evidence_links`, `_structural_issues`, `_visit_coverage_issues`, `_activity_day_gap_issues`	route
11	`protocol_agent.py:1721`	`refine_node`	candidate + audit	repaired `ExtractedSchedule`	audit
12	`protocol_agent.py:1762`	`finalize_node`	candidate	attaches classification + `evidence_facts`, runs `expand_schedule`, sets `verification_*`	END
13	`protocol_extraction.py:718`	`expand_schedule`	schedule	**deterministic** `canonical_plan → flat visits` via `project_canonical_plan`; dedupe; sort; `validate_canonical_plan`	return
14	`server.py:2270`	`extract_protocol_alias` (cont.)	schedules	one Mongo `protocol_extractions` doc per schedule, `expires_at = +2 h`	client

The single most important structural fact: the model authors **one canonical graph**, and the flat visit list is a *pure deterministic Python projection* of it (`expand_schedule:729` — "any model-authored duplicate rows are ignored"). Cycle arithmetic, open-ended bounding and relative-anchor resolution are code, not model output, and are unit-testable with no API key.

Section 5

Extraction Data Model

Two layers. `CanonicalSchedulePlan` (`schedule_schema.py:367`) is the truth; `ExtractedVisit[]` is a compatibility view.

5.1 The canonical plan (v2)

ENTITY	FIELDS THAT MATTER
`ScheduleAnchor:239`	`id, name, anchor_type` ∈ {consent, screening, randomization, first_dose, dose, cycle_start, period_start, last_dose, end_of_treatment, discharge, progression, other}, `source_label`, `evidence_ids`
`SchedulePhase:251`	`phase_type` ∈ {screening, run_in, treatment, washout, follow_up, extension, other}, `parent_phase_id`
`ScheduleBranch:262`	`branch_type` **free text** — arm / period / sequence / cohort / dose_level / sub_study, `parent_branch_id`
`ScheduleEvent:308`	`event_type` (protocol's own code or taxonomy), `phase_id`, `arm_id`, `period_id`, `timing`, `window`, `activity_ids`, `required`, `conditional_text`, `operational_constraints`, `evidence_ids`
`TimingExpression:136`	`kind` ∈ {offset, calendar_offset, range, relative, event_driven, constraint, recurrence, unresolved}, `anchor_id`, `offset`, `range_start/end`, `relation` ∈ {before, after, on, within, between}, `qualifier` ∈ {exact, approximate, minimum, maximum, up_to, as_needed}, `calendar_mode`, `source_label`, `alternative_source_labels`, `weekday_rule`
`WindowSpec:197`	`scope` visit/activity, `window_type` ∈ {tolerance, validity, lookback, minimum_gap, maximum_gap, other}, `state` ∈ {stated, not_stated, unclear, conflicting}, `early`, `late`
`ActivityTemplate:298`	`name`, **`timing`** (intra-day), `window`, `conditional_text`, `operational_constraints`
`RecurrenceRule:333`	`event_ids`, `frequency`, `start_occurrence`, `end_occurrence` (nullable = open-ended), `until_event_id`
`TransitionRule:344`	`from_event_id`, `to_event_id`, `relation`, `amount`
`ScheduleCondition:278`	`expression`, `applies_to_ids`, `occurrence_numbers` (cycles 2/4/6), `applies_to_branch_ids` (factorial)
`ScheduleConflict:358`	`field_path`, `description`, `resolution`, `status`
`SourceEvidence`	`evidence_id, claim, source_location, source_quote, confidence, page_evidence_id`

Two validators deliberately **downgrade rather than reject**: a timing with no offset/anchor becomes `kind="unresolved"` keeping `source_label` (`downgrade_unsupported_shape:157`), and a `stated` window with no magnitude becomes `unclear` (`downgrade_valueless_stated_window:224`). Rejecting either would discard an otherwise usable schedule; inventing a value would breach the no-manufactured-window rule.

5.2 Concept coverage

CONCEPT	REPRESENTED?	FIELD / MODEL	USED DOWNSTREAM?
trial / protocol / protocol version	partial	`ExtractedTrialDetails`, `plan.protocol_id/protocol_version`	yes (trial metadata, version label)
schedule	yes	`CanonicalSchedulePlan`	yes
phase / epoch	yes	`SchedulePhase`	yes → `Epoch`

CONCEPT	REPRESENTED?	FIELD / MODEL	USED DOWNSTREAM?
arm / cohort / part / sequence / substudy	yes (one generic table)	`ScheduleBranch.branch_type`	yes → `StudyDimension` / `GenericDimension`
visit / event	yes	`ScheduleEvent`	yes
event type	yes	`ScheduleEvent.event_type`	yes
timing / timing mode	yes	`TimingExpression.kind` + `calendar_mode`	yes
window	yes	`WindowSpec`	yes
activity	yes	`ActivityTemplate`	yes
activity (intra-day) timing	yes	`ActivityTemplate.timing`	yes
anchor	yes	`ScheduleAnchor`	yes
dependency	yes	`TransitionRule`	yes → `Dependency`
condition	yes	`ScheduleCondition`	**as a blocking qualifier only**
trigger / action	**no**	—	no `ConditionalDefinition` / `ConditionalAction` source
recurrence / repeat rule / stop rule	yes	`RecurrenceRule` (+ `until_event_id`)	yes
applicability	yes (arm/period only)	`event.arm_id` / `period_id`	yes → `ApplicabilityRule`. Country / age / population **not represented**
confinement	implied only	`day_end`, `operational_constraints` free text	**no** — no `ConfinementEpisodeDefinition` source
study day	yes	`anchor_study_day`, `includes_day_zero`, `source_label`	yes
qualifier / footnote	evidence only	`visit_evidence.table_footnotes` (`EvidenceFact`), `conditional_text`	**partially** — no marker, no scope, no meaning
evidence	yes	`SourceEvidence`, `FieldEvidence`	yes
unresolved issue	yes	`assumptions`, `verification_issues`, `canonical_validation`, `ScheduleConflict`	yes
review state	yes	`verification_status/confidence/iterations/scores`, per-visit `review_status`	yes

Actually represented but structurally weaker than UCTSM can hold: footnotes (no `marker`), conditions (no trigger/action/resolution), confinement (none). **Not represented at all:** conditional actions, repeat blocks with a dependency/resume mode, day-numbering convention as a first-class object, visit mode (`visit_mode` / `allowed_visit_modes`).

Section 6

Current Add Trial Flow

`frontend/app/(app)/sponsor/add-trial.tsx`

1. **Protocol ID lookup** → `GET /api/protocols/lookup?protocol_id=` (`server.py:2170`). Tenant-scoped: a registry hit, or a trial the caller can access; never a global directory.
2. **PDF upload** → `POST /api/protocols/extract` (`server.py:2218`), 30-minute client timeout. Returns `details` + `extraction_id` (s). Multi-substudy protocols return `extractions[]` — every SoA is extracted in the one analysis, no pick-then-re-upload (`needs_schedule_selection` is now always `false`).
3. **`submit()`** (`add-trial.tsx:245`):
   - `POST /api/trials` → Mongo `trials` row (`server.py:2374`).
   - for each extraction: `POST /api/trials/{id}/protocol-extractions/{eid}/consume` (`server.py:3976`) → `_persist_schedule_definition` (`server.py:3773`) writes a durable Mongo `schedule_definitions` doc `{canonical_plan, evidence_facts, compatibility_visits, verification, classification, status:'draft_review'}` and sets `trials.current_schedule_definition_id`.
   - `buildCanonicalSchedule(trialId)` → `POST /api/trials/{id}/uctsm-schedule` (`server.py:5818`) → the adapter.
   - navigate to `/(app)/sponsor/protocol-schedule` with the new `schedule_version_id`.
   - **on any failure of the last two steps**, fall through to the legacy editor `/(app)/sponsor/visit-schedule` (`add-trial.tsx:303-308`). The trial and the extraction survive; only the canonical build is lost. **This silent fallback is why B1/B2 have not been noticed: a real protocol lands in the legacy editor and the flow looks like it worked.**

The alternate entry — `POST /api/trials/{id}/extract-schedule` (`server.py:3879`), called from `sponsor/visit-schedule.tsx:735` — re-uploads the PDF for an existing trial, persists a schedule definition, and returns editor rows. It never builds a canonical version.

Section 7

Current Legacy Visit Schedule Architecture

Mongo collections: `visits` (trial-level templates), `visit_instances` (per-patient), `schedule_definitions` (the canonical plan store), `schedule_versions` (legacy snapshot/versioning), `schedule_reviews`, `shares`.

Template model — `VisitIn` (`server.py:352-397`): `visit_number, name, day_offset, day_end, calendar_offset_value/unit, window_days, window_before/after, hour_offset, hour_offset_basis, hour_end, arm/arm_label, period, substudy_label, source_day_label, anchor_study_day, includes_day_zero, relative_to, relative_offset_days, activities, procedures, operational_constraints, visit_type,`

`location, checklist, clinical_tasks, admin_tasks, extraction_warning, review_status, field_evidence`.

It is a genuinely careful flat model — it keeps the protocol's own timing label, the day-numbering convention, asymmetric windows, absolute-vs-within-day hour semantics, and calendar-month approximations so per-patient maths can be redone from a real baseline. What it cannot express: anchors other than baseline, conditions, triggers, recurrence rules, confinement episodes, footnote meaning, activity-level timing as data (only `procedures` free text).

Materialization — `materialize_visit_instances` (`server.py:5007`): idempotent per patient; filters templates by `substudy_label` and `arm_label` (untagged template matches everyone); computes `scheduled_date`, `scheduled_end`, `window_start/end` from `_patient_visit_anchor`; on any arithmetic failure sets `operational_status='manual_review'` with a reason rather than inventing a date; snapshots clinical/admin tasks per instance so a template edit never rewrites history. **Never** infers completion or "missed" from elapsed time.

Legacy authoring routes: `POST/PUT/PATCH/DELETE /api/visits`, `POST /api/trials/{id}/schedule-preview`, `/schedule-definition`, `/api/schedules/{trial}/approve|flag`, `/api/schedule-reviews/*`. All of these are returned `410 Gone` when `UCTSM_AUTHORITATIVE` is truthy (`disable_legacy_schedule_authoring`, `server.py:10268`). **It is not set in `.env` today**, so legacy authoring is live.

Section 8

UCTSM Architecture

`app/domain/schedule/` is a pure domain layer (Pydantic, no I/O); `app/services/` is the transactional layer; `app/api/uctsm.py` is 41 routes.

MODULE	RESPONSIBILITY
`models.py` (633)	`UniversalSchedule` and every child entity + patient-side enums
`timing.py` (231)	16 timing expression types, 3 reference kinds
`evaluator.py` (1262)	`ScheduleEvaluator.evaluate` — the engine
`conditional.py` (284)	`build_conditional_plan` — *what* becomes applicable
`condition.py` (148)	three-valued (`TRUE/FALSE/UNKNOWN`) expression evaluation
`validator.py` (717)	`ScheduleValidator` — the approval gate
`deviation.py` (304)	visit / activity / confinement deviation classification
`diff.py` (307)	version-to-version comparison
`parity.py` (264)	legacy-vs-engine comparison for the cutover
`projection.py` (87)	human-readable protocol schedule rows
`enrolment.py` (175)	which dimensions must be chosen at enrolment
`anchor_status.py` (156)	`AWAITING_EVENT / PLANNED / ACTUAL / CONFIRMED` vocabulary
`exceptions.py`	`ScheduleNotApprovedError`, `UnsupportedTimingError`, `ImmutableScheduleError`

Engine guarantees verified in code (`evaluator.py`):

- `evaluate` refuses a non-APPROVED version (`:779`), refuses a context pinned to a different version (`:781`), and refuses an approved schedule that fails integrity validation (`:784`).
- Events are processed in topological order (`:797`); each event's resolved dates feed `working_context.event_values`, and anchors derived from events are resolved via `derivation_rule.selection` FIRST/LAST (`:815-821`).
- **Missing anchor → `WAITING_FOR_ANCHOR`, never today's date** (`:1181`). `_resolve_reference` returns `None`; there is no `date.today()` fallback anywhere in timing resolution.
- `ApproximateTiming`, `ProtocolDefinedTiming` and `UnresolvedTiming` raise `UnsupportedTimingError` → status `UNRESOLVED` (`:600-602`, `:1177`).
- `ON_DEMAND` events are *available, never due* (`_evaluate_on_demand:137`).
- `ACTUAL_PREVIOUS_EVENT` mode blocks until an actual date exists (`:1140-1156`); `MANUAL` mode waits for a supplied date; default is `NOMINAL`.
- `ActivityReference` resolves only against a **recorded actual** time; a planned time is never substituted (`timing.py:61-68`, `_resolve_reference:492`).
- Confinement is one parent episode with study days; the patient stays `IN_CONFINEMENT` until an actual discharge is recorded even if the planned discharge has passed (`_evaluate_confinement:889`); overlapping episodes are flagged (`_flag_overlapping_episodes:83`).
- Open-ended repeats are bounded by a **rolling horizon** and every rule with more to come emits a `RepeatSummary`, so occurrence N is never presented as the last one (`_apply_rolling_horizon:322`).

Section 9

UCTSM Data Model

39 tables. Relational where identity and joins matter, JSON where the shape is a reviewed document.

GROUP	TABLES
Trial / protocol	`uctsm_trials`, `uctsm_protocols`, `uctsm_protocol_versions`
Extraction	`uctsm_extraction_runs`
Schedule definition	`uctsm_schedule_definitions`, `uctsm_schedule_versions`

GROUP	TABLES
Schedule children	`uctsm_epochs`, `uctsm_arms`, `uctsm_cohorts`, `uctsm_populations`, `uctsm_anchors`, `uctsm_events`, `uctsm_event_applicability`, `uctsm_event_dependencies`, `uctsm_event_recurrence`, `uctsm_activities`
Evidence / review	`uctsm_evidence`, `uctsm_claim_evidence`, `uctsm_validation_issues`, `uctsm_review_decisions`
Patient	`uctsm_patients`, `uctsm_patient_schedule_assignments`, `uctsm_patient_anchors`, `uctsm_patient_states`, `uctsm_patient_conditions`
Patient schedule	`uctsm_schedule_generation_runs`, `uctsm_patient_schedules`, `uctsm_schedule_evaluations`, `uctsm_patient_events`, `uctsm_patient_event_occurrences`, `uctsm_patient_unscheduled_visits`, `uctsm_patient_activities`, `uctsm_patient_activity_records`
Change control	`uctsm_schedule_impact_proposals`, `uctsm_parity_runs`, `uctsm_audit_events`

Stored as JSON on `uctsm_schedule_versions`: `dimensions`, `conditional_definitions`, `repeat_blocks`, `confinement_episodes`. On `uctsm_events`: `timing`, `conditions`, `conditional_actions`, `qualifiers`, `confinement`, `metadata`. On `uctsm_activities`: `timing`, `conditions`, `applicability`, `qualifiers`. So every richer construct **is** persisted; there is no table gap.

No deviation table and no reminder/calendar table. `DeviationService.report` computes deviations on read from patient events + actuals; reminders and calendar entries are pure projections (`schedule_projection.project`). Both are deliberate ("recomputed rather than stored, so the calendar always reflects the current admission and discharge state", `uctsm.py:849`).

Section 10

UCTSM Extraction Flow

Answering the brief's ten questions directly:

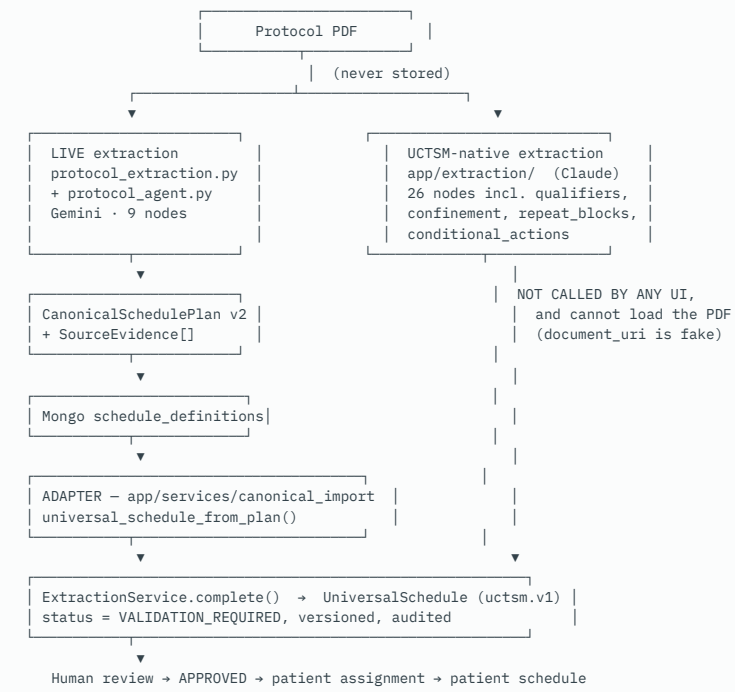
1. **Does UCTSM parse the PDF directly?** Yes — in `app/extraction/runner.py` (`pages_from_pdf`, reusing `protocol_document_index._extract_pdf_text_pages`).
2. **Does it use the current extraction system?** Also yes, by a *different* route: `canonical_import` translates the live pipeline's plan.
3. **Does it have a separate extractor? Yes — a whole second one.** `app/extraction/graph.py::build_extraction_graph` runs **26 sequential nodes**: `document_structure → schedule_discovery → metadata → epochs → dimensions → anchors → events → event_types → timing → conditions → dependencies → dependency_modes → recurrence → repeat_blocks → activities → activity_timing → qualifiers → conditional_actions → conditional_resolution → confinement → relationships → evidence_linking → completeness → consistency → assembly → final_validation`.
4. **Does it receive a canonical plan?** On the import route, yes — a `CanonicalSchedulePlan` dict.
5. **A normalized JSON object?** Yes, on both routes; the native route's provider returns claim dicts.
6. **Already-created legacy visits?** No. `compatibility_visits` are ignored by the adapter, which reads `canonical_plan` only.
7. **Manually built?** Only in `app/services/demo_service.py` and the `/dev/uctsm-workbench` screen.
8. **Which LLM provider?** Native route: **Anthropic Claude** (`app/extraction/llm_client.py::AnthropicClient`, gated on `ANTHROPIC_API_KEY`, `runner.provider_configured`). Import route: whatever the live pipeline used (Gemini).
9. **Which prompt?** `app/extraction/prompts.py` (426 lines) — a prompt per claim category, including the **qualifiers** stage that extracts footnote markers *and what each one means* (`prompts.py:345-362`).
10. **What schema does it expect?** `UniversalSchedule` (`uctsm.v1`), assembled by `ClaudeExtractionProvider.assemble_schedule` and validated at `final_validation`, which additionally runs `ScheduleValidator` **and** `run_completeness_checks(schedule, pages)`.

Connection status of the native pipeline: IMPLEMENTED BUT NOT CONNECTED.

- Endpoint exists: `POST /api/uctsm/protocols/{pid}/versions/{vid}/extract-schedule` (`uctsm.py:487`), 202 + `BackgroundTasks`, polled via `GET /extraction-runs/{id}`.
- Frontend wrappers exist: `queueScheduleExtraction`, `getExtractionRun` (`frontend/src/features/uctsm/api.ts:364, 377`). **Grep confirms nothing calls either.**
- It could not succeed on an Add Trial protocol anyway: `runner.load_pages` resolves `document_uri` through `storage.get_storage().open()`, and `canonical_bridge` writes `private://protocol-extractions/{id}` — a URI no storage backend can open, because the PDF was never saved.
- `run_completeness_checks` (the footnote/marker completeness proof) is wired **only** into this graph. The live import path never runs it.

Section 11

Current Extraction → UCTSM Relationship



The adapter is not hypothetical. `canonical_bridge.import_schedule_definitions` resolves/creates `Trial → Protocol → ProtocolVersion`, is idempotent on `idempotency_key = canonical-import:{external_schedule_definition_id}` so a retried Add Trial reuses the draft a reviewer may already be editing, joins the Mongo definition to the canonical one via `ScheduleDefinition.external_schedule_definition_id`, and **skips a definition with no `canonical_plan` or no events rather than creating an empty approvable schedule.**

Section 12

Adapter / Mapping Analysis

Full mapping performed by `canonical_import.universal_schedule_from_plan`.

A. Direct mappings

LIVE EXTRACTION	UCTSM TARGET	TRANSFORMATION	LOSS RISK
`plan.anchors[]`	`Anchor`	`_ANCHOR_TYPES` lookup; `_code(name)` → stable upper-case code	**HIGH — no BASELINE (B1)**
`plan.phases[]`	`Epoch`	code + `sequence_number` = position	low
`branch_type ∈ arm/cohort/population`	`StudyDimension`	direct	low
any other `branch_type`	`GenericDimension(dimension_type=…)`	slug the term	low — this is what lets period/sequence/substudy/dose-level survive
`events[]`	`Event`	code, labels, `event_type` via `_event_type`, `epoch_id`, `sequence_number`	low
`timing.kind=offset`	`OffsetTiming`	`_amount` + `_signed`	med
`kind=range`	`RangeTiming`	both ends required	low
`relation=within` / `qualifier=up_to`	`WithinTiming`	`direction` BEFORE/AFTER	low
`qualifier=maximum`	`NoLaterThanTiming`		low
`qualifier=approximate`	`ApproximateTiming`		**HIGH — engine raises on it**
`qualifier=as_needed`	`ProtocolDefinedTiming(AS_NEEDED)` + `activation=ON_DEMAND`		low
`window.state=stated`	`Window` + `NominalWindowTiming`	one-sided → 0 on the other side, which is the stated reading	low
`activities[]`	`Activity` (+ `timing`)	`requiredness=CONDITIONAL` if `conditional_text`	low
`recurrences[]`	`Event.recurrence`	`end_occurrence` →COUNT, `until_event_id` →EVENT, else **HORIZON** (never an invented max)	med
`transitions[]`	`Dependency(TEMPORAL)`	keyed on `to_event_id`	med — `relation` and `amount` are dropped
`conditions[]`	`Qualifier(CONDITION, resolved=False)`		**HIGH — blocking, unresolvable (B2)**

LIVE EXTRACTION	UCTSM TARGET	TRANSFORMATION	LOSS RISK
`conflicts[]`	`ValidationIssue(CONFLICTING_EVIDENCE)`	blocking while unresolved	low
`evidence_facts[]`	`Evidence` + `ClaimEvidence`	claim types EVENT_NAME, TIMING, ACTIVITY, ACTIVITY_TIMING, APPLICABILITY, DEPENDENCY, RECURRENCE, QUALIFIER	med — `page_number` is never populated

B. Deliberately refused (correct behaviour, not a defect)

- fractional amounts (`_amount:151` — "half a day is not a schedulable quantity");
- `qualifier=minimum` → `UnresolvedTiming` rather than silently scheduling the earliest permitted date as if it were planned;
- a repeat whose first occurrence is offset from its anchor → blocking `UNSUPPORTED_PROTOCOL_CONSTRUCT` , because placing it on the anchor anyway would move every cycle of a real patient's treatment;
- a window the extractor reported as `unclear` / `conflicting` → a qualifier, never a manufactured tolerance.

C. Information that is lost

LOST	WHERE IT DIES	CONSEQUENCE
Footnote markers ("a", "†")	plan has no marker field; `Qualifier.marker=None`	`qualifier_markers` conflict detection in `validator._clinical_completeness` can never fire; a reviewer sees "Protocol note" not "Footnote a" (`presentation.ts:860`)
Evidence page numbers	`_EvidenceIndex` sets `section_title` / `source_text` / `source_locator` but not `Evidence.page_number`	`formatEvidence` in `ProtocolScheduleScreen` cannot show "Page 42" for imported schedules
`TransitionRule.relation` + `amount`	`Dependency` has no amount field used here	min/max-gap between two visits becomes an untyped ordering edge
`TimingExpression.weekday_rule` , `alternative_source_labels`	never read	"first Monday after" is dropped
`WindowSpec.window_type` (validity / lookback / min-gap)	`_window_for` only produces tolerance	a 28-day *validity* window becomes a ±28-day *tolerance*
`ScheduleCondition.occurrence_numbers` and `applies_to_branch_ids`	read by `project_canonical_plan` for the flat view, **ignored by the adapter**	cycles-2/4/6 imaging and factorial arm scoping do not reach UCTSM as structure
`total_cycles` , `schedule_kind` , `assumptions` , `source_notes` , `verification_*`	not carried into the canonical version	reviewer loses the extraction's own confidence and stated assumptions

D. UCTSM fields with no source in the live plan

`ConditionalDefinition` (trigger/actions/resolution), `ConditionalAction` , `RepeatBlock` (dependency_mode / resume_mode), `ConfinementEpisodeDefinition`

- `ConfinementDayDefinition` , `Event.confinement` , `Event.visit_mode` / `allowed_visit_modes` , `Event.dependency_mode` , `Anchor.derivation_rule` , `ScheduleMetadata.day_numbering` , `Activity.applicability` , `StudyDimension.criteria` / `parent_dimension_type` / `parent_code` .

Verified by grepping `canonical_import.py` : every one of those names appears zero times except `confinement` (once, in a comment).

E. Semantic mismatches

1. **Baseline anchor divergence (B3).** `project_canonical_plan:900` prefers a randomization/first_dose/cycle_start/period_start anchor; `canonical_import:399` uses `anchors[0]` . **PROVEN:** for anchors `[Informed Consent, Randomisation]` and an unanchored "Day 8" row, the legacy flat view counts from Randomisation and UCTSM counts from Informed Consent.
2. **Baseline anchor absence (B1).** `_ANCHOR_TYPES` has no `BASELINE` ; the fallback `Anchor(code="BASELINE", anchor_type="BASELINE")` is created **only when the plan declares no anchors at all** (`canonical_import:404-412`). `OperationalBridgeService.enroll_patient:148` requires one. **PROVEN BY PROBE** — same plan, first anchor renamed:

   ```
   anchor named 'Baseline'      anchors=[('BASELINE','FIRST_DOSE'),('LAST_DOSE','LAST_DOSE')]      → ENROLLED OK
   anchor named 'First Dose'    anchors=[('FIRST_DOSE','FIRST_DOSE'),('LAST_DOSE','LAST_DOSE')]    → ValueError: approved schedule has no baseline ancho
   anchor named 'Randomisation' anchors=[('RANDOMISATION','FIRST_DOSE'),('LAST_DOSE','LAST_DOSE')] → ValueError: approved schedule has no baseline ancho
   ```

   In the live flow this surfaces as `HTTP 409 Canonical schedule assignment failed` , the patient row is left `schedule_assignment_status='error'` , and `_project_operational_enrollment` explicitly **refuses to fall back to the legacy materializer** — so the patient ends up with no schedule at all.
3. **Event-type vocabulary split.** Three vocabularies coexist: `canonical_import._EVENT_TYPES` and `validator.CORE_EVENT_TYPES` agree (`TELEPHONE_CONTACT` , `REMOTE` , `LABORATORY` , `IMAGING` , `INPATIENT_ADMISSION`), but `schedule_projection.PATIENT_WORDING` / `ATTENDANCE_EVENT_TYPES` use `PHONE_CONTACT` , `REMOTE_VISIT` , `LAB_ONLY` , `IMAGING_ONLY` , `PROCEDURE_ONLY` , `INPATIENT_CONFINEMENT` , `ONSITE` . **It fails safe** — an unmatched type gets "Your study activity is scheduled" and `requires_attendance=False` — but a phone call never gets phone-call wording.
4. **`ProjectedEvent.event_type` defaults to `"SITE_VISIT"`** (`schedule_projection.py:55`). The only caller passes it explicitly, so this is latent, not live; it is still a default that says "travel".
5. **`ApproximateTiming` round trip.** The adapter can produce it; the evaluator raises `UnsupportedTimingError` on it. "Approximately Day 30" therefore imports cleanly, approves (it is not a blocking validator issue), and then renders as `UNRESOLVED` for every patient.

Section 13

Schedule Engine Capability Matrix

Against the finalized requirements in the brief.

A. Anchor-based scheduling

REQUIREMENT	STATUS	EVIDENCE
BASELINE	**conflicting**	`models.Anchor.anchor_type` is free text and `BASELINE` is used by the bridge, but `canonical_import._ANCHOR_TYPES` cannot produce it → **B1**
RANDOMIZATION, FIRST_DOSE, LAST_DOSE, EOT, PROGRESSION, DISCHARGE	implemented	`_ANCHOR_TYPES`
SURGERY, WITHDRAWAL	partial	fold into `PROTOCOL_EVENT`; no dedicated type
custom / event reference	implemented	`Anchor.source_event_code` + `derivation_rule.selection`; `EventReference`
no fallback to current date	implemented	`_resolve_reference` returns None; no `date.today()` in timing
missing anchor stays unresolved	implemented	`WAITING_FOR_ANCHOR` (`evaluator.py:1182`)
dependent visits only recalculate	implemented	topological order + `_impact` diff
completed visits protected	implemented	`terminal_statuses={COMPLETED,MISSED,CANCELLED}`, `protected_event_ids` (`impact_service.py:32, 336`)
impact preview	implemented	`PatientScheduleImpactService.preview/confirm/cancel` + 4 preview endpoints
audit	implemented	`PATIENT_ANCHOR_CONFIRMED` vs `PATIENT_ANCHOR_CHANGED` (`impact_service.py:208`)

B. Conditional / triggered visits

Engine: **fully implemented** — `ConditionalDefinition` (condition, trigger_event, applicability, actions, resolution_condition, recurrence), `ConditionalAction.action_type`, `build_conditional_plan`, `ConditionState`, `PatientCondition` table, `POST /patients/{id}/conditions/impact-preview`, `PatientScheduleScreen` conditional-requirements table. Safety rules encoded: an unactivated conditional event gets no date and therefore no overdue (`conditional.py:13-19`); `PENDING_CONFIRMATION` does not act; resolution stops further repeats without deleting history. No automatic clinical detection — correct for Phase 1.

Extraction: MISSING. Nothing produces a `ConditionalDefinition`. The live plan's `ScheduleCondition` becomes an unresolved qualifier instead. So this area is **engine-complete, input-empty.**

C. Repeating / open-ended

Implemented throughout. `RecurrenceRule` with `RecurrenceTermination ∈ {COUNT, DATE, EVENT, CONDITION, HORIZON}`; `HORIZON` is the adapter's default so no arbitrary maximum is invented; `_apply_rolling_horizon` materializes a bounded upcoming set and emits `RepeatSummary` for anything that continues; `OPEN_ENDED_PREVIEW_CYCLES = 4` in the extractor is explicitly a *display preview* ("MTB must never present occurrence N as the last one", `protocol_extraction.py:104-110`); `BlockState` distinguishes pause from stop.

D. Nominal vs actual previous event

Implemented. `DependencyMode ∈ {NOMINAL, ACTUAL_PREVIOUS_EVENT, MANUAL, UNCLEAR}`; `ACTUAL_PREVIOUS_EVENT` blocks until an actual exists; branch-only recalculation via topological order; completed history protected. **Reviewer UI exists and is the one thing `ProtocolReviewScreen` actually does** (`chooseMode`, `DEPENDENCY_MODE_CHOICES`). Extraction never sets it, so the default is `NOMINAL` unless a reviewer chooses — the safe, reviewable outcome.

E. Activity-level / intra-day timing

Implemented. `Activity.timing` + `ActivityReference` (intra-day anchor on another activity's **actual** time), `activity_actual_key(event, occurrence, activity)`, `PatientActivity` / `PatientActivityRecord` tables, `evaluate_activities` with `WAITING_FOR_ANCHOR` + `waiting_for_activity` explanation, minute/hour units in `TimeUnit`. Progressive chains work because each resolved actual feeds the next. The adapter carries `ActivityTemplate.timing` across. **Gap:** `bridge_visit_documents` projects `activities` as **names only** and `procedures/clinical_tasks/admin_tasks` as empty lists, so intra-day timing is invisible in the legacy Mongo view the patient/CRC visit-detail screens read.

F. Multi-day / confinement

Engine implemented: `ConfinementEpisodeDefinition → ConfinementDayDefinition → activity codes`; `ConfinementStatus ∈ {…, IN_CONFINEMENT, EXTENDED, DISCHARGED, WAITING_FOR_ANCHOR, NOT_APPLICABLE}`; admission/dose/discharge stay distinct events so the dosing anchor is never replaced; a conditional extension moves the expected discharge without creating a second episode; overlap flagged; reminder projection absorbs member events so an admitted patient is not told to come in. Readmission is expressible as a second episode. Crossover/washout via `Epoch`/period dimensions. **Extraction: MISSING** — no source for `confinement_episodes`. A multi-day row imports as one event with `day_end`-derived timing, i.e. as a visit, not an episode.

G. Footnotes / qualifiers

PIECE	STATUS
footnote extraction	live pipeline captures them as `table_footnotes` evidence facts only; native pipeline has a real `qualifiers` stage
marker mapping	**missing** — live plan has no marker field → `Qualifier.marker=None`
semantic scope	partial — adapter always uses `QualifierScope.VISIT` / `ACTIVITY`, never GLOBAL/ARM
qualifier classification	partial — only WINDOW and CONDITION categories are produced

PIECE	STATUS
unresolved marker	implemented — becomes blocking `UNRESOLVED_QUALIFIER`
orphan footnote	implemented in `completeness.py` — **not wired to the import path**
source evidence	implemented, minus page number
survival into patient execution	partial — displayed on the schedule table; **not carried into `visit_instances`**
resolution	**missing — no endpoint, no UI → B2**

H. Applicability

Engine: `ApplicabilityRule(dimension, IN/NOT_IN, values, field, condition)`, three-valued evaluation (`UNKNOWN` → `WAITING_FOR_CONDITION`, never a guess), `GenericDimension` for any protocol structure, `parent_dimension_type/parent_code` for "Part A, Cohort 1", `enrolment_options` tells the enroller what must be chosen, `_dimension_id` refuses an ambiguous label rather than picking one. Extraction supplies **arm and period only**. Country, age and population have no source. Compound rules are expressible (`ApplicabilityRule.condition`) but never produced.

I. Protocol versioning

Fully implemented and the strongest area. Immutable versions (`ImmutableScheduleError` on any edit or revalidation of APPROVED/SUPERSEDED); `Patient.current_schedule_version_id` pins the patient; `PatientScheduleAssignment` records every assignment with a reason; `PatientContext` refuses a mismatched version; **approval ≠ in force** — `ScheduleVersion.effective_from` is honoured by `_approved_version`, which refuses to enrol onto a version that takes effect later; no automatic migration (a version change goes through impact preview/confirm); `list_schedule_versions` lists superseded versions with their patient counts so "which schedule is this patient on" stays answerable; `compare_schedule_versions` diffs two versions.

J. Event types

`CORE_EVENT_TYPES` covers SITE_VISIT, TELEPHONE_CONTACT, HOME_VISIT, ASSESSMENT, IMAGING, LABORATORY, INPATIENT_ADMISSION, INPATIENT_DISCHARGE, UNSCHEDULED, SAFETY_ASSESSMENT, TREATMENT, PROCEDURE, REMOTE + the presentation-safe generics VISIT/CONTACT/OTHER. **An unknown type is a non-blocking WARNING, not a block** — deliberate, with the reasoning at `validator.py:40-44`: refusing approval over vocabulary would stop real work for no clinical reason, while presenting an unknown type as an ordinary site visit could tell a patient to travel. Unknown types also fail safe in the reminder projection (§12.E.3). `visit_mode` / `allowed_visit_modes` are never set by the adapter, so a hybrid visit is never flagged for per-patient mode selection on imported schedules.

Section 14

UI / Screen Capability Matrix

TARGET (BRIEF §15)	IMPLEMENTED	WHERE
Protocol schedule: `Visit | Type | Timing | Window | Key Activities | Applies To | Status`	**exact match** (plus a Visit Name column)	`ScheduleTable.tsx:52-59`
Patient schedule: `Visit | Type | Planned Date | Window | Actual Date | Key Activities | Status`	yes (labelled "Expected Date" / "Allowed Window")	same component, `patient` flag
Expanded visit: `Activity | Timing Rule | Planned Time | Actual Time | Status`	yes	`ScheduleTable.tsx:382-393`
Conditional requirements: `Condition | Action | Timing | Status | Confirm`	yes	conditional table + `previewConditionImpact`
Confinement: `Episode → Study Day → Activities`	yes	`ConfinementTable`, `toConfinementRows`
Anchor status	yes (bonus)	`AnchorStatusTable`
Deviations	yes (bonus)	`PatientScheduleScreen:103`
Version context header	yes (bonus)	`VersionContextHeader`
Mobile card fallback	yes	`ScheduleTable.tsx:451-490`
Qualifier resolution	**no — read-only display only**	`toQualifierDetails:857` — `resolved` is rendered, never set
Per-field review	present but a rubber stamp	see below

Navigation is fully wired: `add-trial` → `sponsor/protocol-schedule`; `clinical/trial-summary` → `sponsor/schedule-versions` → `protocol-schedule`; `protocol-schedule` → `clinical/protocol-review`; `pi/dashboard` + `crc/dashboard` → `clinical/action-board` → `clinical/patient-schedule`; `clinical/visit-detail` → `clinical/patient-schedule` (gated on `patient.uctsm_patient_id`).

Finding — the review gate is bypassable from the UI. `ScheduleReviewService.approve` correctly requires a `ReviewDecision` per event per field (`display_name`, `timing`, and `activities` / `conditions` / `applicability` where present). But `ProtocolScheduleScreen.confirmFields` (`:90-105`) loops over **every event and every field** and posts `decision:"CONFIRM"` with the canned comment *"Confirmed from the human-readable protocol schedule."* on a **single button press.** The backend's per-field human-review requirement is satisfied mechanically without anyone having looked at anything, and the audit trail will show a reviewer confirmed every field individually. Severity: **P1 clinical safety / compliance.**

Section 15

Patient Scheduling Flow

```
POST /api/patients                                  server.py:5963
  ├─ UCTSM_AUTHORITATIVE and no uctsm_trial_id → 409 (refuse to enrol)
  ├─ trial.uctsm_trial_id present:
  │    _project_operational_enrollment               server.py:5601
  │      → _enroll_operational_patient_sync (threadpool)
  │        → OperationalBridgeService.enroll_patient
  │            ├─ _approved_version  (APPROVED + effective_from ≤ today)
  │            ├─ find/create uctsm_patients + PatientScheduleAssignment
  │            ├─ resolve arm/cohort via _dimension_id (refuses ambiguity)
  │            ├─ require a BASELINE anchor          ◀── B1 FAILS HERE
  │            ├─ write PatientAnchor(CONFIRMED, source=OPERATIONAL_ENROLMENT)
  │            └─ PatientScheduleService.evaluate(horizon = baseline+366d)
  │      → bridge_visit_documents → upsert Mongo visit_instances
  │         (uctsm_source_of_truth: True, keyed on uctsm_patient_event_id)
  │      → cancel obsolete canonical instances (never completed/cancelled ones)
  │      → patients.schedule_assignment_status = 'assigned'
  │      on failure: status='error', HTTP 409, and NO legacy fallback
  └─ else: materialize_visit_instances (legacy templates)
```

Two safety properties worth naming: a canonical failure **refuses to silently fall back to legacy** (`server.py:5636-5639`), and re-enrolment is idempotent (`idempotency_key = operational-enrolment:{external_patient_id}:{version}`).

Read path after cutover (`server.py:4394-4430`): `GET /api/visits/mine` checks `read_mode_for_trial` ; when `ENGINE` , `apply_engine_dates` overlays **only** `scheduled_date` , `window_start` , `window_end` onto the existing operational documents. Nothing is created or deleted, so a site's comment thread cannot be lost, and flipping back to `LEGACY` is instant and total because the original dates were never overwritten (`operational_read.py:1-19`).

Section 16

Reminder / Calendar / Deviation Flow

```
Anchor / condition / assignment / state change
  → PatientScheduleImpactService.preview   (proposal + expiry + fingerprint)
  → operator confirms                      → .confirm
  → PatientScheduleService.evaluate        → new PatientEvent rows
  → bridge_visit_documents                 → Mongo visit_instances (dates only)
  → GET /api/uctsm/patients/{id}/schedule/projection
       → schedule_projection.project()     → reminders[] + calendar[] + suppressed[]
  → GET /api/uctsm/patients/{id}/deviations → DeviationService.report
```

CONSUMER	STATUS
projection API	implemented — `uctsm.py:809`
reminder delivery	**MISSING ENTIRELY.** No scheduler, no cron, no worker turns a projected reminder into a notification or push. `server.py` startup starts only `_ensure_indexes` , `_migrate_visit_instances` , `_ensure_admin_seed` , `_migrate_organization_ownership` , `broadcast_worker_loop` . `db.notifications` is written only for team events, schedule approve/flag, and shares. `/api/reminders` is **medication** reminders the user creates by hand.
`notification_projection.project_notification_candidate`	**TEST ONLY** — grep shows exactly one reference, in `tests/test_uctsm_notifications.py`
calendar consumer	implemented — `GET /api/calendar/team` (`server.py:8382`) reads `visit_instances` and therefore sees canonical dates; `patient/calendar.tsx` likewise. The UCTSM `calendar[]` projection itself has no consumer.
patient notification of a moved visit	not implemented
CRC notification	partial — `GET /api/uctsm/dashboard/actions` → `ActionBoardScreen` is a pull, not a push
deviation calculation	implemented — `DeviationService.report` , computed on read, surfaced in `PatientScheduleScreen`
audit	implemented — see §18

Section 17

Versioning Architecture

```
Protocol ─┬─ ProtocolVersion (content-addressed by plan_fingerprint)
          │     └─ ScheduleDefinition (name + schedule_type)
          │           ├─ ScheduleVersion v1 APPROVED effective_from 2026-01-01
          │           └─ ScheduleVersion v2 APPROVED effective_from 2026-10-01
          └─ current_version_id (advanced only if previously NULL — an amendment
                                 never silently replaces the version in use)

Patient.current_schedule_version_id  ← pinned at enrolment, changed only through
                                       impact preview → confirm
```

`_protocol_version` de-duplicates on `document_hash` , so two imports of the same plan resolve to the same protocol version instead of stacking. Duplicate `version_label` values are disambiguated (`"v1.0 (2)"`) rather than rejected, because the label is the protocol's own text.

Section 18

Audit Architecture

Two independent audit stores. **This is itself a finding.**

- **Mongo** `audit_logs` via `write_audit(user, action, detail, target_id, trial_id)` — `trial.extract_protocol`, `trial.extract_schedule`, `trial.consume_protocol_extraction`, `trial.uctsm_schedule_build`, `trial.uctsm_link`, `patient.enroll`, `patient.uctsm_reconcile`, …
- **SQL** `uctsm_audit_events` with `organization_id, actor_id, action, entity_type, entity_id, before, after, reason`. 27 actions, including `PATIENT_ANCHOR_CANDIDATE_RECORDED`, `PATIENT_ANCHOR_CANDIDATE_DERIVED_FROM_ACTUAL`, **`PATIENT_ANCHOR_CONFIRMED`** and **`PATIENT_ANCHOR_CHANGED`** (correctly distinguished at `impact_service.py:206-210`), `PATIENT_SCHEDULE_IMPACT_PREVIEWED / _CANCELLED / _STALE`, `PATIENT_SCHEDULE_EVALUATED`, `SCHEDULE_EVENT_CORRECTED`, `SCHEDULE_APPROVED`, `SCHEDULE_VALIDATED`, `OPERATIONAL_READ_MODE_CHANGED`.

Recorded per requirement: old value (`before`), new value (`after`), user (`actor_id`), timestamp, reason, affected visits (inside the impact proposal payload), schedule version, patient, source evidence (via `ClaimEvidence`). **Role is not recorded** on `uctsm_audit_events` — only `actor_id`. Dependent schedule changes are visible, but through the `ScheduleImpactProposal.impact` blob rather than as discrete per-visit rows.

`ReviewDecision` is a separate, richer trail for review: `entity_type, entity_id, field_path, previous_value, new_value, reviewer_id, reason`.

Section 19

Database Architecture

ENTITY	DB	TABLE / COLLECTION	PK	KEY RELATIONSHIPS
Trial (operational)	Mongo	`trials`	`id` (uuid4 str)	`uctsm_trial_id`, `uctsm_schedule_definition_id`, `uctsm_draft/approved_schedule_version_id`, `current_schedule_definition_id`
Trial (canonical)	SQL	`uctsm_trials`	uuid	`organization_id`, `external_trial_id` → Mongo `trials.id`
Protocol	SQL	`uctsm_protocols`	uuid	→ trial; `current_version_id`
Protocol version	SQL	`uctsm_protocol_versions`	uuid	→ protocol; de-duped on `document_hash`
Extracted plan (operational)	Mongo	`schedule_definitions`	`id`	→ trial; `canonical_plan`, `evidence_facts`, `compatibility_visits`
Extraction cache	Mongo	`protocol_extractions`	`id`	TTL 2 h; `user_id`, `trial_id` after consume
Schedule definition	SQL	`uctsm_schedule_definitions`	uuid	→ protocol version; `external_schedule_definition_id` → Mongo `schedule_definitions.id`
Schedule version	SQL	`uctsm_schedule_versions`	uuid	unique (definition, version_number); `extraction_run_id`, `effective_from`
Event / Activity / Anchor / Epoch / dimensions	SQL	`uctsm_*`	uuid	→ schedule version, unique code per version
Condition (definition)	SQL	JSON on `uctsm_schedule_versions.conditional_definitions`	—	—
Confinement (definition)	SQL	JSON on `…confinement_episodes`	—	—
Recurrence	SQL	`uctsm_event_recurrence`	uuid	1-1 with event
Applicability	SQL	`uctsm_event_applicability`	uuid	→ event
Evidence	SQL	`uctsm_evidence` / `uctsm_claim_evidence`	uuid	→ schedule version / claim entity
Patient (operational)	Mongo	`patients`	`id`	`uctsm_patient_id`, `assigned_schedule_version_id`, `uctsm_evaluation_id`, `schedule_assignment_status`
Patient (canonical)	SQL	`uctsm_patients`	uuid	`external_patient_id` → Mongo; `current_schedule_version_id`
Assignment	SQL	`uctsm_patient_schedule_assignments`	uuid	→ patient, version
Patient visit	SQL	`uctsm_patient_events` (+ `_occurrences`, `_unscheduled_visits`)	uuid	`logical_key`, `logical_occurrence_id`
Patient visit (operational)	Mongo	`visit_instances`	`id` = `uctsm_patient_event_id` when canonical	`uctsm_source_of_truth`, `canonical_status`
Patient activity	SQL	`uctsm_patient_activities` / `_activity_records`	uuid	→ patient event
Deviation	—	**none — computed on read**	—	`DeviationService.report`
Reminder	Mongo	`reminders` (**medication only**)	`id`	no visit reminders anywhere
Calendar	—	**none — computed on read**	—	`/calendar/team`, projection
Audit	Mongo + SQL	`audit_logs`, `uctsm_audit_events`	—	two independent trails

Section 20

Legacy vs UCTSM Comparison

CAPABILITY	LEGACY	UCTSM	PRODUCTION CONNECTED?	KEEP	REPLACE
PDF extraction	yes — Gemini + LangGraph, 9 nodes	Claude, 26 nodes, **unreachable**	**Legacy only**	legacy	—
Canonical plan schema	`CanonicalSchedulePlan` v2	`UniversalSchedule`	both	both	—
Protocol schedule store	Mongo `schedule_definitions`	SQL `uctsm_schedule_versions`	both	UCTSM	Mongo → read-through
Visit templates	`visits` + editor	`Event`	both	UCTSM	legacy editor
Patient schedule	`visit_instances`	`PatientEvent`	both; canonical projected into Mongo	UCTSM	legacy materializer
Anchors	baseline only	full	UCTSM	UCTSM	—
Conditions / triggers	none	engine yes, input no	engine only	UCTSM	—
Recurrence	expanded to flat rows	real rule + rolling horizon	both	UCTSM	legacy expansion (keep for preview)
Applicability	`arm_label` + `substudy_label` string match	dimensional	both	UCTSM	legacy
Activity timing	free-text `procedures`	full	UCTSM	UCTSM	—
Confinement	none (`day_end` only)	engine yes, input no	engine only	UCTSM	—
Qualifiers	none	model + blocking, no resolution	**blocked**	UCTSM	—
Versioning	`schedule_versions` snapshots	immutable + pinning + effective_from	UCTSM	UCTSM	legacy
Reminders	medication only	projection, no delivery	**neither**	UCTSM	—
Calendar	`/calendar/team` off `visit_instances`	projection, no consumer	legacy (fed canonical dates)	legacy read + UCTSM dates	—
Deviations	none	yes	UCTSM	UCTSM	—
Audit	`audit_logs`	`uctsm_audit_events`	both	unify	—
UI	`visit-schedule.tsx` (2160 lines)	9 UCTSM screens	both	UCTSM	legacy editor eventually
Cutover gate	—	parity run + read-mode	UCTSM	UCTSM	—

Section 21

Requirement-by-Requirement Gap Analysis

Classification per the brief's nine categories. **Nothing here is called "missing" without having traced the implementation first.**

REQUIREMENT	CLASS
Anchor types, no-date-fallback, unresolved anchors, dependent-only recalc, completed protection, impact preview, anchor audit	**1 — fully implemented and production-connected**
Protocol versioning: immutability, pinning, effective_from, no auto-migration, per-version evaluation	**1**
Nominal vs actual vs manual dependency mode, incl. reviewer UI	**1**
Event-type taxonomy + fail-safe unknown handling	**1**
Deviation calculation and display	**1**
Parity gate + read-mode cutover + instant revert	**1**
Repeating / open-ended with rolling horizon and repeat summaries	**1**
Adapter live extraction → UCTSM draft	**1**, with the three defects B1/B2/B3
UCTSM-native Claude extraction pipeline (26 nodes: qualifiers, confinement, conditional actions, repeat blocks, event types)	**2 — fully implemented, not connected** (no caller, and no stored PDF to read)
Footnote/marker completeness checking (`completeness.py`)	**2**
Reminder projection (`schedule_projection`)	**2** — API + UI display exist, no delivery
`notification_projection`	**TEST ONLY**
Conditional / triggered visits	**5 — model + engine + patient UI exist, extraction input missing**
Multi-day confinement	**5 — same shape**
Footnotes / qualifiers	**3 — partially implemented**: captured, blocking, displayed; not markered, not scoped, **not resolvable**
Applicability beyond arm/period (country, age, population, compound)	**5 — engine ready, no extraction source**

REQUIREMENT	CLASS
Activity-level timing end-to-end	**3** — canonical path complete; the Mongo projection drops it
Baseline anchor contract	**4 — implemented incorrectly** (B1)
Baseline anchor selection agreement between the two projections	**4** (B3)
Per-field human review	**4 — implemented incorrectly in the UI** (bulk confirm)
Qualifier resolution	**8 — missing**
Visit-reminder / patient-notification delivery	**8 — missing**
Evidence page numbers on imported schedules	**8 — missing**
Automatic clinical detection of conditions	**9 — explicitly Phase 1 out of scope**, and correctly absent

Section 22

Test Coverage

84 backend test files. Frontend `frontend/tests/` exists (not audited in depth).

AREA	FILES	FIXTURES
Extraction expansion (offline)	`test_protocol_expansion.py` (27 tests), `test_protocol_pattern_regressions.py` (~35 `project_canonical_plan` cases), `test_schedule_schema_v2.py`, `test_protocol_timing_contract.py`, `test_protocol_invariants.py` (29 tests)	hand-built plans, **no LLM**
Extraction agent	`test_protocol_agent.py`, `test_protocol_agent_routing.py`, `test_protocol_json_response.py`, `test_protocol_document_index.py`	**mocked LLM output**
Adapter	`test_canonical_import.py`, `test_canonical_bridge.py`, `test_canonical_journey.py`	synthetic canonical plans
UCTSM domain	`test_uctsm_domain`, `_schedule_semantics`, `_conditional_engine`, `_condition_workflow`, `_activity_timing`, `_confinement`, `_event_types`, `_generic_dimensions`, `_rolling_horizon`, `_schedule_diff`, `_anchor_status`	in-memory SQLite
UCTSM services / API	`_api_contract`, `_workflow`, `_persistence`, `_impact`, `_assignment_impact`, `_enrolment(+_api)`, `_deviation(+_service)`, `_dashboard`, `_parity`, `_cutover`, `_operational_bridge`, `_operational_projection`, `_projection`, `_notifications`, `_version_effect`, `_unscheduled_api`, `_footnote_completeness`, `_demo_api`	SQLite
UCTSM extraction	`test_uctsm_extraction.py`, `test_uctsm_llm_extraction.py`	**mocked Claude responses**
Legacy Mongo	`test_visit_instances`, `test_schedule_edit`, `test_schedule_definition_api`, `test_calendar`, `test_patient_visit_anchor_safety`, …	**live MongoDB Atlas**, RUN_ID-marked, module teardown

Measured results.

- `pytest -k "uctsm or canonical"` → **340 passed, 0 failed** (35 s).
- `pytest tests/test_canonical_import.py tests/test_canonical_bridge.py tests/test_canonical_journey.py` → **35 passed.**
- Full `pytest tests/` → **590 passed, 68 failed, 232 errors, 1 skipped** (7 m 48 s).
- Diagnosed, not guessed: the red is **test infrastructure, not product.**
  - `tests/test_mtb_backend.py` targets a dead remote preview URL (`BASE = …code-viewer-87.preview.emergentagent.com`) → 23 failures, all 404.
  - Every Mongo test file creates its **own module-level event loop** (`LOOP = asyncio.new_event_loop()`) and there is **no `conftest.py`**. Motor pins its IO loop on first use, so the first module to touch Mongo wins and every later module errors. Verified by re-running in isolation: `test_visit_instances.py` alone → **41 passed**; `test_password_recovery.py` alone → **3 passed**; both fail in a combined run.
- Consequence: **the suite cannot be run as one job.** There is no CI-runnable green state, so a regression in the Mongo half is invisible — and B1 slipped through precisely because the one test that would have caught it uses a fixture anchor literally named "Baseline".

Section 23

Real-PDF Extraction Validation Status

`backend/eval/` has a three-tier harness and an honest `FINDINGS.md`.

- **Tier 1** — `tests/test_protocol_expansion.py`, 27 deterministic tests modelled on real protocols (the PICN collapsed column expanding 9 → 25 visits, cadence changes, open-ended bounding, relative-anchor chains, circular-reference termination, cycles-2/4/6 conditional activities, undated visits sorted last, dedupe, runaway capping, purity). Offline, <1 s. **Passing.**
- **Tier 2** — `eval/invariants.py` + `tests/test_protocol_invariants.py`, 29 structural-coherence tests that fire on any schedule the system ever produces (lost day offsets, unexpanded name templates leaking to the UI, duplicate bookings, non-chronological output, impossible windows, `schedule_kind` contradicting the visit list, screening visits with positive offsets). **Passing.**
- **Tier 3** — `eval/corpus_eval.py`: 63 real protocol PDFs graded by an independent LLM judge that reads the source document. `eval/judge.py:6-7` states plainly: *"no per-file expected output is hand-authored"*.

> `eval/FINDINGS.md`: **"Status: not yet run against the new design."** The configured key returns *400 — credit balance is too low*. The document also retracts an earlier "4/4 archetypes pass" claim, noting it came from four synthetic PDFs generated to match the prompt's own assumptions and *"could not have discovered the collapsed-cycle problem, because no synthetic fixture collapsed a cycle."*

Conclusion: this is an extraction validation gap, not a code implementation gap. No real Schedule-of-Assessments table has been extracted and compared against a manually reviewed expected answer. Tier 3 is blocked on billing, and even once unblocked, an LLM judge without hand-authored ground truth cannot establish per-field accuracy. Known limitations the docs already own: open-ended protocols, degraded scans, truly divergent multi-arm layouts.

Section 24

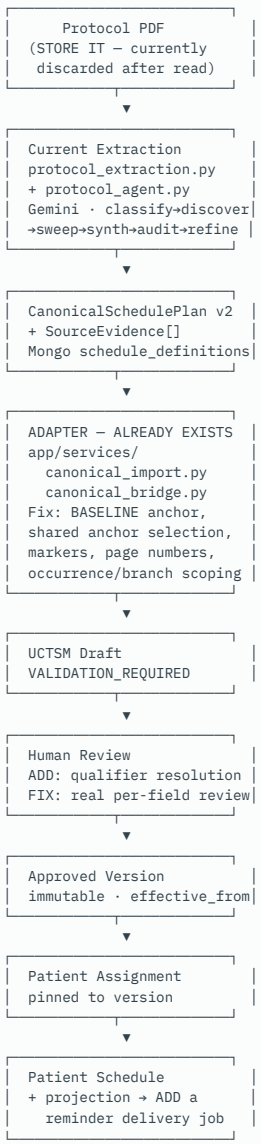
Risks / Conflicts / Duplicated Logic

1. **P0 — B1 baseline anchor.** Enrolment fails for any imported schedule whose first anchor is not literally named "Baseline". Proven. Hidden by the fixture.
2. **P0 — B2 unresolvable qualifiers.** Any footnote, condition or unclear window permanently blocks approval; no resolution path exists in API or UI.
3. **P0 — B3 divergent baseline selection** between `project_canonical_plan` and `canonical_import`. Two different dates for the same protocol row. Proven.
4. **P1 — Bulk field confirmation.** `confirmFields` rubber-stamps the per-field human review the backend requires, and the audit trail will not show it as a bulk action.
5. **P1 — Two extraction pipelines.** ~215 KB of live Gemini pipeline and ~1.5 kLOC of unreachable Claude pipeline, with the *unreachable* one owning the qualifier, confinement, conditional-action and repeat-block stages the product needs. Deciding which one is the future is the single highest-leverage architectural call.
6. **P1 — Test suite cannot run as one job.** No `conftest.py`, per-module event loops, one file pointed at a dead URL, and the Mongo tests write to live Atlas.
7. **P1 — Event-type vocabulary split three ways.** Fails safe today; will not stay safe as types are added.
8. **P1 — No reminder delivery at all.** Visit reminders are the product's core patient-facing promise and no code sends one.
9. **P2 — SQLite in the live path.** `UCTSM_DATABASE_URL=sqlite:///uctsm-dev.db` with `UCTSM_DEMO_MODE=true` bypasses Alembic via `create_all`. 13 migrations exist and are unexercised in this environment.
10. **P2 — Two audit trails.** A change that crosses the boundary is half in Mongo and half in SQL; no correlation id joins them, and role is not recorded on the SQL side.
11. **P2 — 2-hour extraction TTL.** A sponsor who starts Add Trial, is interrupted, and returns loses a 30-minute extraction.
12. **P2 — Duplicated day arithmetic.** `schedule_schema.project_canonical_plan` and `app/domain/schedule/evaluator.evaluate_timing` independently compute dates from the same plan. The parity gate exists precisely because of this, which is the right mitigation but not a fix.
13. **P2 — 481 KB `server.py`.** Route/helper coupling makes the Mongo↔SQL boundary hard to see and hard to test.
14. **P2 — `.env` contains live secrets** (Gemini, Anthropic, Brevo, JWT, Mongo) and `DEV_OTP_MODE=true` with a fixed code. Outside audit scope but must not ship.

Section 25

Recommended Target Architecture

The proposed architecture in the brief is confirmed correct. `?????` replaced by what the code actually shows:



Recommendation on the two pipelines: keep the **live Gemini pipeline** as the production extractor and **port the missing stages into its canonical plan schema** (`qualifiers` with markers, `confinement_episodes` , `conditional_definitions` , `repeat_blocks` , `visit_mode`). Reasons: it is the one that runs; it has the page-chunk full-coverage guarantee and the PDF context cache; its output already round-trips through a deterministic, offline-testable projection; and its provider is configured and funded. Retire `app/extraction/` 's provider and graph but **salvage `prompts.py` (the qualifiers / confinement / event_types prompt text) and `completeness.py` (the marker-completeness proof) into the live pipeline** — those two files are the valuable part of that subsystem.

Section 26

Exact Missing Connections

#	MISSING CONNECTION	PRECISE LOCATION
MC-1	No `BASELINE` anchor is ever produced from a plan that declares anchors	`canonical_import._ANCHOR_TYPES:98` ↔ `operational_bridge.enroll_patient:148`
MC-2	No qualifier-resolution endpoint	`app/api/uctsm.py` — zero occurrences of `qualifier`
MC-3	No qualifier-resolution UI	`presentation.toQualifierDetails:857` reads `resolved` , nothing writes it
MC-4	The two projections do not share baseline-anchor selection	`schedule_schema.project_canonical_plan:900` ↔ `canonical_import:399`
MC-5	`run_completeness_checks` not wired to the import path	called only at `app/extraction/graph.py:115`
MC-6	`project_notification_candidate` has no caller	`app/services/notification_projection.py:17`
MC-7	No job converts a projected reminder into a notification / push	nothing between `schedule_projection.project` and `db.notifications`
MC-8	Native extraction endpoint has no caller	`frontend/src/features/uctsm/api.ts:364, 377`
MC-9	Protocol PDF never stored, so `document_uri` is unresolvable	`server.py:2244` (read, discarded) ↔ `canonical_bridge:176` ↔ `runner.load_pages:90`
MC-10	`Evidence.page_number` never populated on import	`canonical_import._EvidenceIndex:66-80`

#	MISSING CONNECTION	PRECISE LOCATION
MC-11	Activity timing / procedures not projected into `visit_instances`	`operational_projection.bridge_visit_documents:57-64`
MC-12	`occurrence_numbers` / `applies_to_branch_ids` read by the flat projection, ignored by the adapter	`schedule_schema:938` ↔ `canonical_import:497`
MC-13	Reminder-projection event-type vocabulary does not match the canonical one	`schedule_projection.py:30-49` ↔ `validator.CORE_EVENT_TYPES:45`
MC-14	No `conftest.py`; no single runnable suite	`backend/tests/`

Section 27

Implementation Order

Strictly dependency-ordered; each step independently shippable.

Phase 0 — unblock the production path (P0).

1. **MC-14 first.** `conftest.py`, one event loop, an isolated test database, delete or re-point `test_mtb_backend.py`. Do this *before* the fixes below — without it there is no way to prove a fix.
2. MC-1: give the adapter an explicit BASELINE anchor contract, and change the journey-test fixture so its anchors are **not** named "Baseline".
3. MC-4: one shared baseline-anchor selection function used by both projections.
4. MC-2 + MC-3: qualifier resolution, API then UI, recorded as a `ReviewDecision`.
5. Replace the bulk `confirmFields` rubber stamp with per-row confirmation.
6. **In parallel, non-engineering:** unblock the extraction eval billing and hand-author ground truth for a 10-protocol subset. This is procurement and human review time; everything downstream is guesswork without it.

Phase 1 — close the Phase-1 workflow (P1). 7. MC-9: store the protocol PDF; give `ProtocolVersion.document_uri` a real key. 8. MC-10: carry evidence page numbers through the adapter. 9. MC-5: run completeness checks on the import path. 10. MC-7 + MC-6: a reminder delivery job built on `project` + `reconcile`. 11. MC-13: one event-type vocabulary; drop the `SITE_VISIT` default. 12. Port the qualifier/marker stage into the live extraction schema.

Phase 2 — richness (P2). 13. MC-12: occurrence and branch scoping into the adapter. 14. `confinement_episodes` extraction + import. 15. `conditional_definitions` extraction + import. 16. MC-11: project activity timing into `visit_instances`. 17. `visit_mode` / `allowed_visit_modes` extraction. 18. PostgreSQL + Alembic in every environment; retire the SQLite `create_all` path. 19. Unify the two audit trails behind one correlation id; record role.

Phase 3 — retire. 20. Set `UCTSM_AUTHORITATIVE=1` per trial after its parity run passes. 21. Remove `app/extraction/`'s provider and graph, keeping `prompts.py` and `completeness.py`. 22. Make `sponsor/visit-schedule.tsx` read-only, then delete it.

Section 28

Files That Would Need Modification

FILE	CHANGE	PHASE
`backend/tests/conftest.py`	**new** — shared loop, isolated DB	0
`backend/tests/test_mtb_backend.py`	re-point at ASGITransport or delete	0
`backend/tests/test_canonical_journey.py`	fixture anchors must **not** be named "Baseline"	0
`backend/app/services/canonical_import.py`	BASELINE anchor; shared anchor selection; markers; page numbers; occurrence/branch scoping; confinement + conditional import	0-2
`backend/schedule_schema.py`	extract the shared baseline-anchor selector; add a marker field to the footnote/qualifier structure	0-1
`backend/app/services/operational_bridge.py`	accept the explicit baseline contract	0
`backend/app/api/uctsm.py`	qualifier-resolution route	0
`backend/app/db/repositories.py`	persist a resolved qualifier	0
`backend/app/services/schedule_service.py`	qualifier resolution as a `ReviewDecision`	0
`frontend/src/features/uctsm/api.ts`	qualifier-resolution call	0
`frontend/src/features/uctsm/ProtocolReviewScreen.tsx`	qualifier resolution UI	0
`frontend/src/features/uctsm/ProtocolScheduleScreen.tsx`	replace bulk confirm with per-row confirmation	0
`frontend/src/features/uctsm/presentation.ts`	qualifier rows become actionable	0
`backend/eval/corpus_eval.py`, `backend/eval/FINDINGS.md`	hand-authored ground truth for a 10-protocol subset	0 (parallel)
`backend/server.py`	store the uploaded PDF; pass a real `document_uri`; extend the extraction TTL	1
`backend/app/services/canonical_bridge.py`	real `document_uri`	1
`backend/app/extraction/completeness.py`, `graph.py`	expose completeness for reuse by the import path	1

FILE	CHANGE	PHASE
`backend/app/services/notification_projection.py`	give it a caller	1
`backend/app/services/schedule_projection.py`	one event-type vocabulary; drop the `SITE_VISIT` default	1
`backend/protocol_agent.py`, `backend/protocol_extraction.py`	qualifier/marker, confinement, conditional-action, visit-mode stages	1-2
`backend/app/services/operational_projection.py`	project activity timing + procedures	2
`backend/.env`, `backend/app/db/base.py`	PostgreSQL + Alembic everywhere	2

Appendix

FINAL GAP TABLE

REQUIREMENT	CURRENT IMPLEMENTATION	ACTUAL LOCATION	STATUS	MISSING CONNECTION	RECOMMENDED ACTION	PRIORITY
Patient enrolment onto an imported schedule	Requires an anchor coded/typed BASELINE, which the adapter never produces	`operational_bridge.py:148` ↔ `canonical_import.py:98`	**4 — implemented incorrectly**	MC-1	Explicit baseline contract in the adapter; fix the misleading fixture	P0
Footnote / qualifier resolution	Blocking issue raised, never resolvable	`validator.py:400`; no writer anywhere	**8 — missing**	MC-2, MC-3	Resolution API + UI recorded as a `ReviewDecision`	P0
Agreement on the baseline anchor	Two independent selections	`schedule_schema.py:900` ↔ `canonical_import.py:399`	**4 — implemented incorrectly**	MC-4	One shared selector	P0
Real per-field human review	One button confirms every field of every event	`ProtocolScheduleScreen.tsx:90`	**4 — implemented incorrectly**	—	Per-row confirmation; never bulk	P1
Real-PDF extraction accuracy	Tier 3 never run; no hand-authored ground truth	`eval/FINDINGS.md`, `eval/judge.py:6`	**extraction validation gap**	—	Unblock billing; ground-truth 10 protocols	P1
Visit-reminder delivery	Projection only, no sender	`schedule_projection.py`; `notification_projection.py:17`	**2 / test-only**	MC-6, MC-7	Delivery job with reconciliation	P1
Protocol PDF retention	Discarded after extraction	`server.py:2244`	**8 — missing**	MC-9	Store it; real `document_uri`	P1
Evidence page traceability	`page_number` never set on import	`canonical_import.py:66`	**8 — missing**	MC-10	Carry the page number through	P1
Footnote completeness proof	Implemented, unwired to the live path	`completeness.py`; `graph.py:115`	**2 — not connected**	MC-5	Run it on the import path	P1
One event-type vocabulary	Three vocabularies coexist	`schedule_projection.py:30` ↔ `validator.py:45`	**3 — partial**	MC-13	Single source of truth	P1
Single runnable test suite	Per-module loops, dead URL, live Atlas	`backend/tests/`	**4 — implemented incorrectly**	MC-14	`conftest.py` + isolated DB	P1
UCTSM-native extraction	26-node Claude pipeline, no caller, unreadable document	`app/extraction/`	**2 — not connected**	MC-8, MC-9	Salvage `prompts.py` + `completeness.py`; retire the rest	P1
Conditional / triggered visits	Engine + UI complete, no extraction input	`conditional.py`, `models.py:304`	**5 — model exists, behaviour unreachable**	—	Extraction stage + adapter mapping	P2
Multi-day confinement	Engine + UI complete, no extraction input	`evaluator.py:853`, `models.py:339`	**5**	—	Extraction stage + adapter mapping	P2
Cycle-specific / factorial condition scoping	Read by the flat view only	`schedule_schema.py:938`	**3 — partial**	MC-12	Map into UCTSM conditions	P2
Activity timing in the operational view	Names only	`operational_projection.py:57`	**3 — partial**	MC-11	Project timing + procedures	P2
Applicability: country / age / population / compound	Engine ready, no source	`models.py:260`	**5**	—	Extraction stage	P2
Visit mode	Never set on import	`canonical_import.py` (absent)	**5**	—	Extraction stage	P2
PostgreSQL in production	SQLite + `create_all` bypasses Alembic	`app/db/base.py:24`; `.env`	**3 — partial**	—	Postgres + Alembic everywhere	P2
Unified audit	Two independent trails; no role on the SQL side	`audit_logs` + `uctsm_audit_events`	**3 — partial**	—	Correlation id + role	P2
Anchor types SURGERY / WITHDRAWAL	Folded into `PROTOCOL_EVENT`	`canonical_import.py:98`	**3 — partial**	—	Add explicit types	P3
Automatic clinical detection of conditions	Absent by design	—	**9 — out of scope**	—	none	P3

Appendix

THE FIFTEEN QUESTIONS

Q1 — What exactly happens today from PDF upload to extracted schedule? `add-trial.tsx:209` posts the PDF to `POST /api/protocols/extract` (`server.py:2218`). That calls `pe.extract_protocol_bundle_all` → `GeminiProtocolExtractor.extract_bundle_all` → a LangGraph agent: classify → discover → evidence_sweep (one parallel Gemini call per 22-page chunk with 4 pages overlap, so every page is read) → synthesize → audit → up to 2 refine loops → finalize. Finalize attaches the classification and evidence facts and runs `expand_schedule`, which deterministically projects the canonical graph into flat visit rows in Python. One `protocol_extractions` Mongo document is written per independent Schedule of Assessments, with a 2-hour TTL. Elapsed time for a large protocol: minutes; the client allows 30.

Q2 — What exact canonical object comes out of extraction? `ExtractedSchedule` (`protocol_extraction.py:419`), whose payload of record is `canonical_plan: CanonicalSchedulePlan` (`schedule_schema.py:367`, `schema_version="2.0"`) — anchors, phases, branches, events, activities, recurrences, transitions, conditions, conflicts — plus `evidence_facts: list[SourceEvidence]`, `classification`, `visits` (the deterministic flat projection), `assumptions`, `canonical_validation` and `verification_*`.

Q3 — Where is that object stored? Twice in Mongo, then once in SQL. `db.protocol_extractions` (transient, 2 h), then `db.schedule_definitions` (durable, `status='draft_review'`, written by `_persist_schedule_definition`, `server.py:3773`, and pointed at by `trials.current_schedule_definition_id`), then translated into `uctsm_schedule_versions` and its child tables. **The PDF itself is stored nowhere.**

Q4 — How does Add Trial consume it? `submit()` creates the trial, POSTs `/protocol-extractions/{eid}/consume` for each extraction to promote the cache into a durable schedule definition, then calls `buildCanonicalSchedule` → `POST /trials/{id}/uctsm-schedule`, then routes to `sponsor/protocol-schedule` with the returned `schedule_version_id`. On any failure of the last two steps it silently falls back to the legacy editor.

Q5 — How does the legacy Visit Schedule get created? Two ways. Either the sponsor saves editor rows through `POST/PUT /api/visits` into `db.visits`, or `POST /api/trials/{id}/extract-schedule` returns extracted rows for review and the sponsor saves them. At enrolment, `materialize_visit_instances` (`server.py:5007`) creates one `visit_instances` document per matching template, filtered by `substudy_label` and `arm_label`, dated from `_patient_visit_anchor`, with `manual_review` on any arithmetic failure rather than an invented date.

Q6 — How does UCTSM currently receive schedule information? Three routes, only one of which is live:

1. **Live:** `POST /trials/{id}/uctsm-schedule` → `canonical_bridge.import_schedule_definitions` → `canonical_import.extraction_result_from_plan` → `ExtractionService.complete`.
2. **Implemented, unconnected:** its own Claude extraction via `POST /api/uctsm/protocols/{pid}/versions/{vid}/extract-schedule` — no caller, and it cannot read the document because the PDF was never stored.
3. **Demo only:** `DemoService` / `/dev/uctsm-workbench`.

Q7 — Can the existing extraction output be directly converted into UCTSM? Yes, and it already is. `universal_schedule_from_plan` performs the full translation, and `test_canonical_import` / `_bridge` / `_journey` (35 tests) pass. The conversion is lossy in the specific ways listed in §12.C and broken in the three ways listed in §12.E, but the mechanism is real and working.

Q8 — Exactly what would the adapter need to map? The adapter needs *fixing*, not building. Concretely: produce a BASELINE anchor (MC-1); share the baseline-anchor selector with the flat projection (MC-4); carry footnote markers and evidence page numbers (MC-10); map `ScheduleCondition.occurrence_numbers` and `applies_to_branch_ids` into UCTSM conditions rather than dropping them (MC-12); carry `TransitionRule.relation` and `amount`; distinguish `WindowSpec.window_type` instead of flattening everything to a tolerance; refuse or downgrade `ApproximateTiming` rather than emitting a shape the evaluator rejects. Then, as extraction gains the stages: populate `confinement_episodes`, `conditional_definitions`, `repeat_blocks`, and `visit_mode`.

Q9 — Which UCTSM capabilities already exist and should be reused? The whole engine. `ScheduleEvaluator` (anchors, dependencies, dependency modes, recurrence, rolling horizon, activities, intra-day chains, confinement), `build_conditional_plan`, `ScheduleValidator`, `ScheduleReviewService` (validate / correct / submit / approve / reject with immutability and per-field review), `PatientScheduleImpactService` (preview / confirm / cancel with completed- visit protection), `DeviationService`, `AnchorStatusService`, `DashboardService`, `compare_schedule_versions`, `ParityService` + read-mode cutover, `OperationalBridgeService`, `bridge_visit_documents`, `apply_engine_dates`, and all nine UCTSM screens.

Q10 — Which finalized requirements are genuinely missing? Only these: qualifier resolution (API + UI); visit-reminder delivery; evidence page numbers on import; protocol PDF retention. Everything else is either implemented and connected, or implemented with an empty extraction input.

Q11 — Which requirements are implemented but disconnected from the live workflow? The 26-node Claude extraction pipeline; `run_completeness_checks`; `project_notification_candidate` (test-only); the UCTSM reminder/calendar projection's *calendar* output; `ConditionalDefinition`, `RepeatBlock` and `ConfinementEpisodeDefinition` (engine + UI + persistence exist, nothing produces them); applicability beyond arm/period; `visit_mode`.

Q12 — Which legacy components can eventually be retired? `db.visits` + `materialize_visit_instances`; the legacy authoring routes already gated by `disable_legacy_schedule_authoring`; `db.schedule_versions` and `db.schedule_reviews`; the 2160-line `sponsor/visit-schedule.tsx` editor; `ClaudeProtocolExtractor` / `OpenRouterProtocolExtractor` if Gemini is confirmed; `app/extraction/`'s provider + graph (keeping `prompts.py` and `completeness.py`). Retire only after each trial's parity run passes and `UCTSM_AUTHORITATIVE` is on — and keep `visit_instances` permanently, because it carries operational state (comments, task completion, who did what) the engine has no equivalent for and should not acquire.

Q13 — What must change to make the production flow work? Phase 0 of §27, in order: fix the test harness so a fix can be proved, then MC-1, MC-4, MC-2/MC-3, then replace the bulk confirm. That is roughly four backend touch-points and three frontend ones. The pipeline, the adapter and the engine stay as they are.

Q14 — Are there any places the current implementation could silently lose clinical scheduling information? Yes. Ranked by clinical risk:

1. **Footnote markers** — a marker is never carried, so the validator's own marker-conflict detection can never fire, and the reviewer sees "Protocol note" instead of "Footnote a". *Silent.*
2. **Window type** — a 28-day *validity* window ("labs valid within 28 days") becomes a ±28-day visit *tolerance*. That is a materially wrong permission. *Silent.*

3. **Cycle-specific and factorial condition scoping** — cycles-2/4/6 imaging and factorial arm restrictions survive into the flat legacy view but not into UCTSM. *Silent.*
4. **Baseline anchor divergence (B3)** — the same "Day 8" row resolves to two different dates depending on which projection you read. *Silent.*
5. **`ApproximateTiming`** — imports and approves, then renders `UNRESOLVED` for every patient. *Visible, but only at the patient screen, after approval.*
6. **Intra-day activity timing** — present canonically, dropped by `bridge_visit_documents`, so the CRC/patient visit-detail screens never show it. *Silent.*
7. **Confinement** — a multi-day row becomes a visit rather than an episode, so an admitted patient can be reminded to travel on an internal study day. *Silent* (mitigated only where a real `ConfinementEpisodeDefinition` exists, which imports never create).
8. **Transition amounts** — a stated minimum gap between two visits becomes an untyped ordering edge. *Silent.*
9. **`weekday_rule`** / **`alternative_source_labels`** — "the first Monday after" is dropped. *Silent.*
10. **Version information** — `verification_*`, `assumptions` and `source_notes` do not reach the canonical version, so the reviewer loses the extractor's own stated uncertainty at the moment they are asked to approve. *Silent, and the most consequential of the "soft" losses.*

Not losses (worth stating, because they look like ones): `WAITING_FOR_ANCHOR`, `UNRESOLVED`, unresolved qualifiers and `unclear` windows are all deliberate, visible refusals to guess.

Q15 — What must NOT be rebuilt because it already exists? The Gemini extraction pipeline and its graph; `CanonicalSchedulePlan` v2 and `project_canonical_plan`; `expand_schedule` and its 27+29 offline tests; `canonical_import` + `canonical_bridge` (the adapter); the entire `app/domain/schedule` engine; `ScheduleValidator`; the review/approval lifecycle; versioning with `effective_from` and patient pinning; `PatientScheduleImpactService`; `DeviationService`; `ParityService` and the read-mode cutover with instant revert; `OperationalBridgeService` and the Mongo projection; and all nine UCTSM screens, whose tables already match the finalized UI spec column for column.

Appendix

PROPOSED IMPLEMENTATION PLAN — FOR APPROVAL ONLY

Nothing below has been implemented.

1. **Exact files to change** — see §28.
2. **Exact modules to reuse** — see Q9 and Q15. No new engine, no new extractor, no new adapter.
3. **Exact adapter required** — none new. `canonical_import.py` needs six targeted changes: BASELINE anchor, shared anchor selection, markers, page numbers, occurrence/branch scoping, `ApproximateTiming` handling.
4. **Database changes** — none required for Phase 0. Phase 1 adds a storage key for the protocol PDF (a field on the Mongo `schedule_definitions` document and a real `ProtocolVersion.document_uri`; no new table). Phase 2 moves `UCTSM_DATABASE_URL` to PostgreSQL and runs the 13 existing migrations.
5. **API changes** — Phase 0 adds exactly one route: `POST /api/uctsm/schedule-versions/{id}/qualifiers/{qualifier_id}/resolve`, which records a `ReviewDecision`, updates the qualifier in place, and re-validates. Phase 1 adds no public route; the reminder job is internal.
6. **UI changes** — Phase 0: qualifier resolution in `ProtocolReviewScreen` (the screen already exists and already carries "the decisions that block approval"), and per-row confirmation replacing `confirmFields` in `ProtocolScheduleScreen`. No new screens.
7. **Migration / compatibility** — additive throughout. Existing drafts keep working; a draft blocked on qualifiers becomes unblockable rather than changing meaning. The BASELINE fix must be **forward-compatible with already- imported versions**, which are immutable once approved: the resolution is to accept a designated baseline anchor by `anchor_type` precedence rather than by name, so existing correct schedules keep working and existing broken ones start working without re-import.
8. **Testing** — `conftest.py` first. Then: a regression test for each of B1/B2/B3 using anchors deliberately *not* named "Baseline"; a test that an imported schedule with a footnote can be resolved and approved; a test that the two projections agree on the baseline anchor for the same plan; and a test that per-field confirmation cannot be satisfied in bulk.
9. **Rollback** — every Phase 0 change is behind existing seams. The baseline fix is a pure function change with no data migration. Qualifier resolution is additive. The read-mode switch already provides instant, total rollback of the cutover itself, and `UCTSM_AUTHORITATIVE` stays off until parity passes.
10. **Legacy deprecation** — per §27 Phase 3, per trial, gated on a passing parity run. `visit_instances` is never retired; it remains the operational record.

Recommendation: approve Phase 0 as a single work item. It is small — roughly four backend files and three frontend files — and it is the difference between a production path that silently falls back to the legacy editor and one that actually completes.

Method code read + targeted execution probes **Changed** nothing — audit only **Source** docs/mtb-uctsm-forensic-audit.md